

Trabalho Prático 2 — Projetando um Gerenciador de Memória Virtual

Disciplina: Sistemas Operacionais **Instituição:** PUC Minas **Autores:** Caio Felix, Milena Cardoso
Repositório (GitHub público): <https://github.com/milenacrd/trabalho-SO-2> **Data:** Junho de 2026

Sumário

1. [Introdução e fundamentação teórica](#)
 2. [Especificação do problema](#)
 3. [Arquitetura e estruturas de dados](#)
 4. [Implementação e decisões de projeto](#)
 5. [Dificuldades encontradas](#)
 6. [Testes realizados e análise dos resultados](#)
 7. [Conclusão](#)
 8. [Uso de Inteligência Artificial \(metodologia SDD\)](#)
 9. [Referências](#)
-

1. Introdução e fundamentação teórica

A **memória virtual** é uma técnica de gerência de memória que desacopla o espaço de endereçamento **lógico** (visão do processo) do espaço de endereçamento **físico** (memória RAM efetivamente instalada). Esse desacoplamento permite que um processo enxergue um espaço de endereçamento contíguo e maior do que a memória física disponível, delegando ao sistema operacional a tarefa de manter em memória apenas as porções de fato utilizadas.

A forma mais difundida de implementar memória virtual é a **paginação**. Nela, o espaço lógico é dividido em blocos de tamanho fixo chamados **páginas**, e a memória física é dividida em blocos de mesmo tamanho chamados **quadros** (**frames**). A tradução de um endereço lógico para físico é feita por uma **tabela de páginas**, que mapeia cada número de página para o número do quadro em que ela reside. Como a tabela de páginas fica na memória, cada tradução exigiria um acesso adicional à memória; para mitigar esse custo, o hardware mantém uma cache associativa de traduções recentes, o **TLB** (**Translation Lookaside Buffer**).

Este trabalho consiste em **simular**, em linguagem C, todo o processo de tradução de endereços de um espaço virtual de $2^{16} = 65.536$ bytes, contemplando:

- a quebra do endereço lógico em **número de página** e **deslocamento** (**offset**);
- a consulta hierárquica **TLB** → **tabela de páginas** → **falta de página**;
- a **paginação por demanda** (**demand paging**), em que uma página só é trazida da memória de retaguarda (`BACKING_STORE.bin`) quando efetivamente referenciada;
- a **substituição de páginas** por um algoritmo **LRU aproximado** baseado em **aging** com bits de referência, necessário porque a memória física (32.768 bytes) é menor que o espaço virtual (65.536 bytes);

- a **política FIFO** de substituição de entradas do TLB;
- a coleta das **estatísticas** de desempenho (taxa de faltas de página e taxa de acerto do TLB).

1.1 Conceitos-chave

- **Página e quadro:** unidades de alocação de tamanho fixo (256 bytes neste projeto). A página pertence ao espaço lógico; o quadro, ao físico.
- **Tradução de endereços:** um endereço lógico de 16 bits é dividido em um número de página (8 bits superiores) e um deslocamento (8 bits inferiores). O endereço físico é $\text{quadro} \times \text{tamanho_do_quadro} + \text{deslocamento}$.
- **TLB hit / TLB miss:** acerto/erro na cache de traduções. Em caso de acerto, o quadro é obtido sem consultar a tabela de páginas.
- ****Falta de página (*page fault*):**** ocorre quando a página referenciada não está residente na memória física, exigindo sua leitura da memória de retaguarda.
- **Localidade de referência:** tendência de um programa acessar, em curtos intervalos de tempo, posições de memória próximas entre si. É o que torna o TLB e os algoritmos de substituição eficazes.
- **LRU e LRU aproximado:** o algoritmo **Least Recently Used** substitui a página cujo último acesso é o mais antigo. Como manter **timestamps** exatos é caro, usa-se a aproximação por **aging**: cada página possui um contador de envelhecimento de 8 bits no qual, periodicamente, desloca-se o conteúdo para a direita e insere-se o bit de referência no bit mais significativo. A página com **menor contador** é a candidata à substituição.

2. Especificação do problema

O simulador foi desenvolvido a partir da especificação fornecida pelo professor, cujos parâmetros são fixos e estão centralizados em `include/config.h`:

Parâmetro	Valor
Espaço de endereçamento virtual	65.536 bytes (2^{16})
Tamanho da página	256 bytes
Entradas na tabela de páginas	256
Tamanho do quadro	256 bytes
Número de quadros	128
Memória física	32.768 bytes
Entradas no TLB	16
Memória de retaguarda	BACKING_STORE.bin (65.536 bytes)

Estrutura do endereço lógico (16 bits úteis):

31	16		15	8		7	0
(ignorado)			número página			offset	
			(8 bits)			(8 bits)	

Requisitos funcionais:

10. Ler inteiros de 32 bits da entrada padrão e considerar apenas os 16 bits menos significativos.
11. Extrair número da página e deslocamento.
12. Traduzir o endereço consultando, nesta ordem: TLB, tabela de páginas e, em último caso, tratando a falta de página.
13. Implementar paginação por demanda lendo páginas de 256 bytes do `BACKING_STORE.bin`.
14. Gerenciar quadros livres e aplicar substituição por **LRU aproximado** (*aging*) quando não houver quadros livres.
15. Invalidar a entrada da tabela de páginas e a entrada correspondente no TLB quando uma página for removida da memória física.
16. Substituir entradas do TLB por **FIFO**.
17. Imprimir, para cada endereço, o endereço lógico, o endereço físico e o byte armazenado.
18. Ao final, relatar a **taxa de faltas de página** e a ****taxa de acerto do TLB****.

Requisitos não funcionais:

- Linguagem **C (padrão C11)**; compilação sem erros nem *warnings*.
- Não é necessário dar suporte a escrita no espaço lógico (somente leitura).
- O `BACKING_STORE.bin` deve ser tratado como arquivo de acesso aleatório (`fopen/fseek/fread/fclose`).

3. Arquitetura e estruturas de dados

O projeto segue uma organização **modular**, separando interface (`include/`) de implementação (`src/`). Cada subsistema do gerenciador de memória é um módulo independente com responsabilidade única.

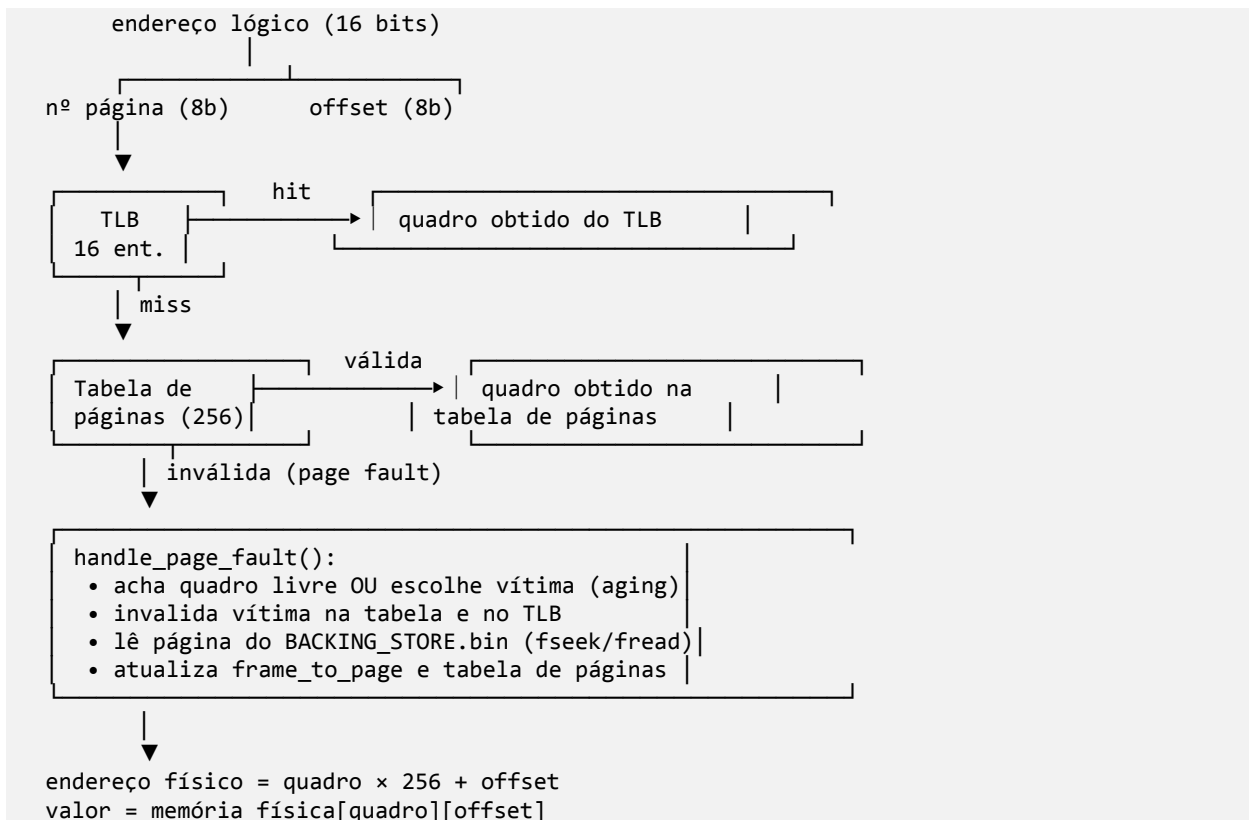
```

vm_manager/
├── Makefile                # regras de compilação (gcc, C11, -Wall -Wextra)
├── README.md
├── include/                # interfaces (headers)
│   ├── config.h           # parâmetros fixos da simulação
│   ├── tlb.h              # interface do TLB
│   ├── page_table.h       # interface da tabela de páginas
│   ├── memory.h           # interface da memória física / paginação
│   └── statistics.h       # interface das estatísticas
├── src/                   # implementações
│   ├── main.c             # laço principal e orquestração da tradução
│   └── tlb.c              # TLB com substituição FIFO

```

—	page_table.c	# tabela de páginas + aging (LRU aproximado)
—	memory.c	# memória física, page fault e substituição
—	statistics.c	# coleta e relatório de estatísticas
—	data/	
—	generate_data.py	# gera BACKING_STORE.bin e arquivos de endereços
—	report/	# este relatório e as evidências de teste

3.1 Visão geral do fluxo de tradução



3.2 Estruturas de dados

Tabela de páginas (`page_table.c`) — vetor de 256 entradas indexado diretamente pelo número da página:

```
typedef struct {
    int frame;                // quadro associado (-1 se inválida)
    int valid;                // 1 se a página está residente
    unsigned char reference_bit; // bit de referência (R)
    unsigned char aging_counter; // contador de envelhecimento (8 bits)
} page_table_entry_t;
```

TLB (`tlb.c`) — vetor de 16 entradas com ponteiro circular para a política FIFO:

```
typedef struct {
    int page;
    int frame;
    int valid;
} tlb_entry_t;
```

```
static int fifo_next; // próxima posição a ser substituída (FIFO)
```

Memória física (`memory.c`) — matriz de 128 quadros × 256 bytes e um vetor de mapeamento inverso quadro → página, usado para localizar quadros livres e para liberar o quadro de uma vítima:

```
static signed char physical_memory[NUM_FRAMES][FRAME_SIZE]; // 128 × 256
static int frame_to_page[NUM_FRAMES];                       // -1 = livre
```

O tipo `signed char` foi escolhido deliberadamente para a memória física (ver Seção 4.4).

Estatísticas (`statistics.c`) — três contadores inteiros: total de endereços traduzidos, faltas de página e acertos do TLB.

4. Implementação e decisões de projeto

A ordem de implementação seguiu a sugestão da especificação e está refletida no histórico de **commits** do repositório (fases 3.1 a 3.5): tradução de endereços → tabela de páginas e **aging** → memória física, **page fault** e substituição → TLB com FIFO → estatísticas.

4.1 Extração de página e deslocamento (`main.c`)

Cada inteiro lido é mascarado com `0xFFFF` para reter apenas os 16 bits inferiores. O número da página são os 8 bits superiores e o deslocamento, os 8 inferiores:

```
logical_address &= 0xFFFF;
int page = (logical_address >> 8) & 0xFF;
int offset = logical_address & 0xFF;
```

O endereço físico é calculado como `frame * FRAME_SIZE + offset`.

4.2 Consulta hierárquica (`main.c`)

O laço principal implementa exatamente a hierarquia da Figura 2 da especificação:

```
int frame = tlb_lookup(page);           // 1) TLB
if (frame != -1) {
    count_tlb_hit();
} else {
    frame = page_table_lookup(page);     // 2) tabela de páginas
    if (frame == -1) {
        count_page_fault();
        frame = handle_page_fault(page); // 3) falta de página
    }
    tlb_insert(page, frame);             // atualiza TLB no miss
}
```

Decisão: o TLB só é atualizado em caso de **miss** (tanto quando a tradução é resolvida pela tabela quanto após uma falta de página), pois em caso de acerto a entrada já existe. Isso mantém a coerência com a política FIFO descrita adiante.

4.3 Tabela de páginas e **aging** (`page_table.c`)

A tabela é indexada diretamente pelo número da página (acesso $O(1)$). As funções principais:

- `page_table_lookup`: retorna o quadro se a entrada for válida, ou `-1`.
- `page_table_update`: associa página $\rightarrow$ quadro, marca como válida e zera o bit R.
- `page_table_invalidate`: invalida a entrada e zera contador/bit (usada na substituição).
- `page_table_set_reference`: marca o bit R da página acessada.
- `page_table_update_aging`: aplica o algoritmo de envelhecimento.

O **algoritmo de aging** é o coração do LRU aproximado:

```
for (int i = 0; i < PAGE_TABLE_SIZE; i++) {
    if (!page_table[i].valid) continue;
    page_table[i].aging_counter >>= 1;           // 1) desloca à direita
    if (page_table[i].reference_bit)
        page_table[i].aging_counter |= 0x80;    // 2) insere R no bit + significativo
    page_table[i].reference_bit = 0;             // 3) zera R
}
```

Decisão (intervalo de atualização): a especificação permite atualizar os contadores "a cada intervalo de atualização (por exemplo, a cada acesso à memória)". Optou-se por atualizar **a cada acesso**, garantindo a melhor aproximação possível do LRU. A sequência no laço principal é primeiro marcar a referência da página acessada e, em seguida, envelhecer todas as páginas:

```
page_table_set_reference(page);
page_table_update_aging();
```

Como consequência elegante dessa ordem, a página recém-acessada termina o ciclo com o bit mais significativo do contador em 1 (valor $\geq 0x80$), ou seja, com o **maior** contador — exatamente o comportamento esperado de "usada mais recentemente", o que a torna a **menos** provável de ser escolhida como vítima.

4.4 Memória física, paginação por demanda e substituição (memory.c)

`handle_page_fault` executa os passos:

19. Procura um quadro livre (`frame_to_page[i] == -1`).
20. Se não houver, chama `select_victim_page` para escolher a vítima.
21. **Invalida** a vítima na tabela de páginas (`page_table_invalidate`) e **remove** sua entrada do TLB (`tlb_remove`), liberando o quadro.
22. Posiciona o arquivo com `fseek(backing, page * PAGE_SIZE, SEEK_SET)` e lê 256 bytes com `fread` para o quadro escolhido.
23. Atualiza `frame_to_page` e a tabela de páginas (`page_table_update`).
24. Retorna o número do quadro.

A **seleção da vítima** implementa o LRU aproximado escolhendo a página válida de **menor** contador de envelhecimento; em caso de empate, vence o menor índice de página (critério determinístico e estável):

```
for (int page = 0; page < PAGE_TABLE_SIZE; page++) {
    if (!page_table_is_valid(page)) continue;
    unsigned char aging = page_table_get_aging_counter(page);
    if (victim_page == -1 || aging < min_aging) {
```

```

    victim_page = page;
    min_aging   = aging;
}

```

Decisão (tipo `signed char`): o `BACKING_STORE.bin` é lido em uma matriz de `signed char`. Como o gerador preenche o arquivo com `byte[i] = i % 256`, valores acima de 127 aparecem como **negativos** na saída — comportamento idêntico ao da solução de referência clássica do problema e coerente com o fato de `char` ser, por padrão, sinalizado na maioria das plataformas. A leitura final do `byte` é trivialmente `physical_memory[frame][offset]`.

4.5 TLB com política FIFO (`tlb.c`)

A inserção no TLB segue três casos, nesta ordem de prioridade:

- 25. **Página já presente:** apenas atualiza o quadro (evita entradas duplicadas).
- 26. **Existe entrada inválida:** ocupa a primeira entrada livre.
- 27. **TLB cheio:** substitui a entrada apontada por `fifo_next` e avança o ponteiro circularmente (`fifo_next = (fifo_next + 1) % TLB_SIZE`).

A função `tlb_remove` invalida a entrada de uma página específica e é chamada sempre que essa página é despejada da memória física, **garantindo que o TLB nunca contenha uma tradução obsoleta** (requisito explícito da especificação).

4.6 Estatísticas (`statistics.c`)

São mantidos três contadores. As taxas são calculadas como proporção sobre o total de endereços traduzidos:

- **Taxa de faltas de página** = `page_faults / total_addresses`
- **Taxa de acerto do TLB** = `tlb_hits / total_addresses`

A saída final segue o formato já previsto no projeto-base, com as taxas impressas com três casas decimais.

5. Dificuldades encontradas

- **Garantir a coerência entre TLB e tabela de páginas:** o ponto mais sensível foi assegurar que, ao despejar uma página, **ambas** as estruturas fossem invalidadas. Sem `tlb_remove` na substituição, o TLB poderia devolver um quadro que já pertence a outra página, gerando traduções incorretas. A invariante de validação (Seção 6) foi essencial para detectar esse tipo de erro.
- **Evitar despejar a página recém-trazida:** era preciso garantir que a página que acabou de causar a falta não fosse imediatamente escolhida como vítima. A ordem "marcar referência → envelhecer" resolve isso naturalmente, pois eleva o contador da página atual ao máximo do ciclo.
- **Sinalização do `char`:** a aparição de valores negativos na saída inicialmente parecia um erro, mas é o comportamento correto para bytes > 127 lidos em `signed char` — confirmado pela invariante `valor == (signed char) offset`.
- **Acesso aleatório ao arquivo:** foi necessário tratar o `BACKING_STORE.bin`

com `fseek/fread` em vez de leitura sequencial, posicionando corretamente o ponteiro em `page * 256` a cada falta de página.

6. Testes realizados e análise dos resultados

6.1 Metodologia de teste

Os dados de entrada são gerados deterministicamente pelo script `data/generate_data.py` (semente fixa `SEED = 42`), o que torna todos os resultados **reproduzíveis**. São gerados:

- `BACKING_STORE.bin` — 65.536 bytes, com `byte[i] = i % 256`;
- `addresses_random.txt` — 10.000 endereços uniformemente aleatórios;
- `addresses_location.txt` — 10.000 endereços com **localidade de referência** (85% dos acessos permanecem em páginas próximas).

A compilação foi verificada com `gcc -Wall -Wextra -O2 -std=c11`, ****sem erros e sem warnings****.

6.2 Validação funcional (oráculo de correção)

A forma como o `BACKING_STORE.bin` é construído fornece um **oráculo de correção** automático e forte. Como `byte[i] = i % 256`, o valor armazenado em qualquer endereço lógico é:

$\text{valor}(\text{endereço}) = (\text{página} \times 256 + \text{offset}) \% 256 = \text{offset}$

Portanto, **para todo endereço**, devem valer simultaneamente:

28. `valor_lido == (signed char) offset`; e

29. `endereço_físico % 256 == offset`.

Um verificador em Python aplicou essas duas condições a **todas as 10.000 traduções de cada cenário**. Resultado (arquivo [`resultados/validacao.txt`](#)):

Cenário	Traduções	Divergências	Resultado
<code>addresses_random.txt</code>	10.000	0	✓ Aprovado
<code>addresses_location.txt</code>	10.000	0	✓ Aprovado

A ausência total de divergências comprova que a tradução de endereços, a paginação por demanda, a substituição de páginas e a coerência TLB/tabela estão **funcionalmente corretas**.

6.3 Estatísticas de desempenho

Saídas completas das estatísticas em [`resultados/estatisticas\\_random.txt`](#) e [`resultados/estatisticas\\_location.txt`](#).

Métrica	Aleatório	Localidade
Endereços traduzidos	10.000	10.000

Métrica	Aleatório	Localidade
Faltas de página	4.958	1.164
Taxa de faltas de página	0,496	0,116
Acertos do TLB	654	7.925
Taxa de acerto do TLB	0,065	0,792

6.4 Análise dos resultados

Os números confirmam, de forma quantitativa, o **princípio da localidade de referência** e a eficácia das estruturas implementadas:

- **Acesso aleatório (sem localidade):** com 128 quadros para 256 páginas possíveis e acessos uniformemente distribuídos, aproximadamente metade das referências cai em páginas não residentes — daí a alta taxa de faltas (**49,6%**). O TLB, com apenas 16 entradas e referências "saltando" por todo o espaço de endereçamento, quase não consegue reaproveitar traduções, resultando em taxa de acerto baixíssima (**6,5%**). Esse é o **pior caso** para qualquer hierarquia de memória.
- **Acesso com localidade:** quando 85% dos acessos se concentram em páginas vizinhas, o conjunto de trabalho (*working set*) cabe folgadoamente nos 128 quadros e, sobretudo, nas 16 entradas do TLB. A taxa de faltas cai para **11,6%** ($\approx 4,3\times$ menor) e a taxa de acerto do TLB salta para **79,2%** ($\approx 12\times$ maior). As faltas remanescentes são majoritariamente **compulsórias** (primeiro acesso a cada página) e decorrentes das trocas periódicas de região do gerador.
- **Efeito combinado TLB + LRU aproximado:** a alta taxa de acerto do TLB no cenário com localidade demonstra que o algoritmo de *aging* mantém em memória exatamente as páginas mais recentemente usadas, que são também as mais prováveis de serem novamente referenciadas. Em outras palavras, a aproximação do LRU por contadores de 8 bits é suficiente para capturar a localidade temporal sem o custo de armazenar *timestamps*.

A diferença gritante entre os dois cenários — partindo do mesmo programa, mesmo **BACKING_STORE** e mesmo número de acessos — é a evidência empírica de que o simulador reage corretamente às características da carga de trabalho.

6.5 Como reproduzir

```
cd data && python3 generate_data.py && cd .. # gera os dados (SEED=42)
make # compila (sem warnings)
./vm < data/addresses_random.txt # cenário aleatório
./vm < data/addresses_location.txt # cenário com localidade
```

7. Conclusão

O trabalho atingiu plenamente seus objetivos: foi construído um simulador funcional de gerenciador de memória virtual que traduz endereços lógicos de 16 bits para endereços físicos utilizando **TLB, tabela de**

páginas e paginação por demanda, com substituição de páginas por LRU aproximado (*aging*) e substituição de TLB por FIFO.

A correção foi comprovada por um oráculo automático que validou **20.000 traduções** (dois cenários × 10.000 endereços) sem nenhuma divergência, e o programa compila sem erros nem **warnings**. As estatísticas coletadas — taxa de faltas de página e taxa de acerto do TLB — não apenas atendem ao requisito de saída, como evidenciam de forma didática o impacto da **localidade de referência** no desempenho de um sistema paginado: a taxa de faltas caiu de 49,6% (acesso aleatório) para 11,6% (acesso com localidade), enquanto a taxa de acerto do TLB subiu de 6,5% para 79,2%.

Do ponto de vista de aprendizado, o projeto consolidou a compreensão dos mecanismos de tradução de endereços, do papel do TLB como cache de traduções e das estratégias de substituição que aproximam o LRU sem o custo proibitivo de manter históricos completos de acesso.

8. Uso de Inteligência Artificial (metodologia SDD)

Em conformidade com a Seção 9 da especificação, esta seção documenta de forma transparente o uso de ferramentas de Inteligência Artificial (IA) no projeto.

8.1 Como a IA foi utilizada

A IA foi empregada como **par de programação** e ferramenta de apoio, nas seguintes frentes:

- **Geração de código** dos módulos a partir de especificações precisas;
- **Explicações** de conceitos (algoritmo de **aging**, semântica do `signed char`, acesso aleatório a arquivo);
- **Depuração** de inconsistências (coerência TLB/tabela de páginas);
- **Testes e validação** — construção do oráculo de correção e análise das estatísticas.

A responsabilidade técnica pelo código entregue é integralmente do grupo: todas as decisões foram revisadas, compreendidas e validadas, e os integrantes são capazes de explicar e defender cada parte da solução.

8.2 Metodologia: Spec-Driven Development (SDD)

O desenvolvimento seguiu o modelo **Spec-Driven Development**, no qual cada incremento parte de uma **especificação** clara antes de qualquer geração de código. Cada **prompt** utilizado procurou:

- definir claramente o problema a ser resolvido;
- especificar requisitos funcionais e não funcionais;
- indicar restrições de implementação (linguagem C11, interfaces fixadas nos **headers**, parâmetros de `config.h`, somente leitura no espaço lógico);
- evitar solicitações vagas ou genéricas.

Essa metodologia se reflete diretamente no histórico de **commits**, organizado em fases incrementais (3.1 a 3.5), cada uma correspondendo a uma especificação abaixo.

8.3 Prompts utilizados (especificações por fase)

Os **prompts** abaixo seguem o formato SDD adotado no projeto. Caso o grupo tenha registrado verbatim **prompts** adicionais usados durante o desenvolvimento, eles devem ser anexados a esta subseção.

Fase 3.1 — Extração de página/offset e tradução (em `main.c`)

Em C (C11), implemente no laço principal de `main.c` a extração do número de página e do deslocamento a partir de um inteiro lido da entrada padrão. Requisitos: considerar apenas os 16 bits menos significativos (máscara `0xFFFF`); número de página = 8 bits superiores; offset = 8 bits inferiores; endereço físico = `frame * FRAME_SIZE + offset`. Restrições: usar os parâmetros de `config.h`; manter o formato de saída "Logical address: %d Physical address: %d Value: %d"; não dar suporte a escrita.

Fase 3.2 — Tabela de páginas e **aging (`page_table.c`)**

Implemente uma tabela de páginas de 256 entradas indexada pelo número da página, com os campos `frame`, `valid`, `reference_bit` e `aging_counter` (8 bits). Funções requeridas (assinaturas fixadas em `page_table.h`): `lookup`, `update`, `invalidate`, `set_reference` e `update_aging`. O `update_aging` deve, para cada página válida: deslocar o contador uma posição à direita, inserir o bit de referência no bit mais significativo (`0x80`) e zerar o bit de referência. Validar índices fora de `[0, 256)`.

Fase 3.3 — Memória física, **page fault e substituição (`memory.c`)**

Implemente a memória física como matriz `signed char[128][256]` e o tratamento de falta de página com paginação por demanda lendo do `BACKING_STORE.bin` via `fopen/fseek/fread`. Em `handle_page_fault`: procurar quadro livre; se não houver, escolher a página vítima com **menor **aging_counter**** (LRU aproximado), invalidar a vítima na tabela de páginas e removê-la do TLB, liberar o quadro, carregar a página correta (`fseek(page*256) + fread(256)`), atualizar o mapeamento quadro→página e a tabela de páginas. Em caso de empate na vítima, escolher o menor índice de página.

Fase 3.4 — TLB com política FIFO (`tlb.c`)

Implemente um TLB de 16 entradas (`page`, `frame`, `valid`). `tlb_lookup` retorna o quadro de uma entrada válida ou `-1`. `tlb_insert` deve: atualizar o quadro se a página já estiver presente; senão usar a primeira entrada inválida; senão substituir por **FIFO** com ponteiro circular. `tlb_remove` invalida a entrada de uma página específica (usada quando a página é despejada da memória física), garantindo que o TLB nunca devolva uma tradução obsoleta.

Fase 3.5 — Estatísticas (`statistics.c`)

Implemente contadores para total de endereços traduzidos, faltas de página e acertos do TLB. Ao final, imprima a **taxa de faltas de página** (`page_faults / total`) e a **taxa de acerto do TLB** (`tlb_hits / total`) com três casas decimais, mantendo o formato de saída do projeto-base.

Apoio à validação e ao relatório

Construa um verificador que confirme, para toda a saída do simulador, que o valor lido é igual ao offset interpretado como `signed char` e que `endereço_físico % 256 == offset`, dado que o `BACKING_STORE.bin` foi gerado com `byte[i] = i % 256`. Em seguida, execute os cenários aleatório e com localidade, colete as estatísticas e analise os resultados.

8.4 Responsabilidade técnica

Conforme a Seção 9.3 da especificação, o grupo é integralmente responsável pelo código entregue; o uso de IA **não substituiu** o entendimento do projeto, e todos os integrantes são capazes de explicar e defender as decisões implementadas.

9. Referências

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 30.ed. (capítulos de Memória Virtual e Paginação).
- TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. 4. ed. (Algoritmos de substituição de páginas; *Aging*).
- Enunciado do Trabalho Prático 2 — *Projetando um Gerenciador de Memória Virtual* (PUC Minas, Sistemas Operacionais).